React - Redux

- What You Actually Need For 3 Years Level
  - React (Must Be Strong)
  - Redux (Interview Level, Not Expert Level)
  - Most Common React + Redux Interview Questions
  - Week Survival Plan
    - Week 1 → React reinforcement
    - Week 2 → Add Redux Toolkit
  - React (Must Be Strong)
    - useState (Very Simple Mental Model)
    - useEffect (Dependencies Made Easy)
    - Controlled Components
      - Controlled Input
      - Uncontrolled Input
    - Lifting State Up
    - Props vs State
    - Re-render Behavior (Most Important)
    - Key Prop in Lists
    - Memoization (Performance Control)(React.memo, useMemo, useCallback)
      - React.memo
      - useMemo
      - useCallback
    - Component Lifecycle (Hooks Version)
    - Context API
      - The problem it solves (prop drilling)
    - Custom Hooks
      - What is a custom hook?
      - The problem it solves
      - Simple example — useFetch
      - Rules (same as all hooks)
  - Redux
    - What Problem Redux Solves
    - Global state vs local state
    - Store
    - Slice
    - Action
    - Reducer
    - Dispatch
    - useSelector
    - useDispatch
    - We will build a simple Cart example
  - Async with createAsyncThunk
  - Most Common React + Redux Interview Questions

What happens when you call dispatch?
What is immutability and why important?
How to avoid unnecessary re-renders?
How to structure a React project?
Typical React Project Structure
components/
Pages/
store/
services/
hooks/
utils/
index.js
App.js

What You Actually Need For 3 Years Level

React (Must Be Strong)

If you're weak here, Redux won't save you.

You must clearly explain:

- `useState`
- `useEffect` (dependencies behavior)
- Controlled components
- Lifting state up
- Props vs state
- Re-render behavior
- Key prop in lists
- Memoization (`React.memo`, `useMemo`, `useCallback`)
- Component lifecycle (mount/update/unmount)

If they ask:

> Why does this component re-render?

You must answer confidently.

Redux (Interview Level, Not Expert Level)

You must understand:

- What problem Redux solves

- Global state vs local state
- Store
- Slice
- Action
- Reducer
- Dispatch
- useSelector
- useDispatch
- Async with `createAsyncThunk`

That's it.

If they ask:

> Why use Redux instead of useState?

Correct reasoning:

- Shared state across many components
- Predictable state transitions
- Centralized logic
- Debugging (Redux DevTools)
- Middleware

If you say "because it's better" → fail.

Most Common React + Redux Interview Questions

Be ready for these:

- What happens when you call dispatch?
- What is immutability and why important?
- How to avoid unnecessary re-renders?
- How to structure a React project?

Week Survival Plan

Week 1 → React reinforcement

- Build small app:

- Login
- Dashboard
- Fetch API
- Loading + error
- Pagination

No Redux yet.

Week 2 → Add Redux Toolkit

- Add auth slice
- Add products slice
- Add async thunk
- Add global loading state

Now you can survive.

> Brutal Truth
>
> At 3 years:
>
> - They don't expect architecture genius.
> - They expect confidence + clean reasoning.
> - If you panic on basic hooks → rejection.

React (Must Be Strong)

If you're weak here, Redux won't save you.

You must clearly explain:

- `useState`
- `useEffect` (dependencies behavior)
- Controlled components
- Lifting state up
- Props vs state
- Re-render behavior
- Key prop in lists
- Memoization (`React.memo`, `useMemo`, `useCallback`)

- Component lifecycle (mount/update/unmount)

useState (Very Simple Mental Model)

```
const [count, setCount] = useState(0);
```

Think like this:

- `count` → current value
- `setCount()` → tells React: “something changed → re-render”

Important rules:

✔ When state changes → component re-renders
✔ State updates are async
✔ React compares old value vs new value
✔ If value is same → no re-render

Example:

```
setCount(0); // if count already 0 → no re-render
```

⚠️ If you mutate object:

```
user.name = "Ali";
setUser(user); // ❌ same reference → no re-render
```

Correct:

```
setUser({ ...user, name: "Ali" }); // ✅ new reference
```

React cares about **reference**, not deep equality.

useEffect (Dependencies Made Easy)

Structure of `useEffect`

```
useEffect(() => {

  // effect code

  return () => {
    // cleanup code
  };

}, []);
```

React interprets it as:

```
run effect
store returned function
```

```
call it later to clean things
```

```
useEffect(() => {
  console.log("runs");
}, [count]);
```

Think:

- No dependency → runs after every render
- `[]` → runs only once (on mount)
- `[count]` → runs when `count` changes

Why infinite loop happens?

```
useEffect(() => {
  setCount(count + 1);
}, [count]);
```

Because:

- count changes
- effect runs
- setCount triggers render
- count changes again
- loop

Controlled Components

Controlled Input

```
<input value={name} onChange={e => setName(e.target.value)} />
```

What is happening?

1. User types `"A"`
2. `onChange` runs
3. `setName("A")`
4. React updates state
5. React re-renders
6. Input value becomes `name`

The value always comes from **React state**.

So React is the boss.

If state = `"Hello"` → input shows `"Hello"`
If state = `""` → input empty

You cannot type without React knowing.

Why does it re-render?

Because:

```
setState() → component re-renders
```

Every key press updates state.

State update = re-render.

That's it. No magic.

Uncontrolled Input

```
<input ref={inputRef} />
```

No `value`.
No state.

User types → browser stores it internally.

React does nothing.

If you want the value:

```
inputRef.current.value
```

DOM is the boss.

The Core Difference

Controlled	Uncontrolled
React stores value	DOM stores value
Uses state	Uses ref
Re-renders on every change	No re-render
Easier validation	Less control

Interview Version (clean answer)

A controlled component is an input where React manages the value via state. Every change updates state and triggers a re-render.

> An uncontrolled component lets the DOM manage the input value using refs, and it does not re-render on every change.

Key memory trick

	Controlled	Uncontrolled
Managed by	React state	The DOM
Value read via	`useState`	`useRef`
Re-renders on type	Yes	No
Real-time validation	Easy	Hard

Lifting State Up

If 2 components need same data:

Instead of:

```
Child A → has state
Child B → needs state
```

You move state to parent:

```
Parent
 ├── Child A
 └── Child B
```

Parent passes data via props.

That's **lifting state up.**

Props vs State

State	Props
Internal	External
Can change	Read-only
Causes re-render	Causes re-render

If parent changes prop → child re-renders.

Re-render Behavior (Most Important)

A component re-renders when:

1. Its state changes
2. Its props change
3. Its parent re-renders

Very important:

Parent re-render → child re-renders
Even if props didn't change (unless memoized)

Key Prop in Lists

```
{users.map(user => (
  <User key={user.id} />
))}
```

What is `key`?

`key` is how React **tracks each item in a list** between renders.

React does not compare full components deeply.
It compares **keys first**.

Think of it like:

> "Which item is the same as before?"

What happens without keys?

React sees:

```
<User />
<User />
<User />
```

It cannot know which one is which.

So it guesses by position.

That's dangerous.

Why `key={index}` is bad

Example:

Initial list:

```
[A, B, C]
```

Rendered:

```
0 → A
1 → B
2 → C
```

Now remove A:

New list:

```
[B, C]
```

If key = index:

```
0 → B
1 → C
```

But React thinks:

- Old 0 (A) became B
- Old 1 (B) became C

It reuses wrong components.

State inside components gets mixed.

Inputs can swap values.

UI bugs appear.

This is not theory. It happens.

Why `key={user.id}` is correct

Because:

- IDs don't change
- IDs stay attached to the same user
- React can correctly match old vs new elements

React sees:

```
Old key 5 → still key 5
Old key 8 → still key 8
```

So it updates only what changed.

`key={index}` **is safe ONLY if:**

1. The list is **static**
2. Items are **never reordered**
3. Items are **never inserted in the middle**
4. Items are **never deleted**
5. The list does not contain **stateful child components**

If any of those conditions are false → index becomes dangerous.

> **The Real Concept**
>
> React uses something called the **reconciliation algorithm.**
>
> Very simplified:
>
> Compare previous tree vs new tree
>
> Use key to match elements
>
> Update only what changed
>
> If keys are wrong → matching is wrong → updates are wrong.
>
> **Interview Answer (clean)**
>
> The key prop helps React's reconciliation algorithm identify which list items changed, were added, or removed. Using a stable unique key like an ID ensures correct component matching. Using index as key can cause incorrect state preservation when the list order changes.
>
> Using index as key is safe only when the list is static and will never change in order or length. Otherwise, it can cause incorrect component matching and state bugs.

Memoization (Performance Control)(`React.memo` , `useMemo` , `useCallback` **)**

All memoization in React solves **one thing**:

> **Avoid unnecessary re-renders or recalculations.**

When a parent re-renders →
ALL its children re-render by default.

Even if their props did NOT change.

```
function Parent() {
  const [count, setCount] = useState(0);

  return (
    <>
      <button onClick={() => setCount(count + 1)} />
      <Child name="Mouad" />
    </>
  );
}
```

When you click button:

- Parent re-renders
- Child also re-renders
- Even though `"Mouad"` never changed

Why?

Because React does NOT automatically compare props.

It just runs the child function again.

This is the key misunderstanding

React by default does NOT skip child rendering
just because props are the same.

It simply re-executes the child component.

What React.memo does

```
export default React.memo(Child);
```

Now React does:

1. Parent re-renders
2. React compares old props vs new props
3. If same → skip child render

Now it actually skips.

Without React.memo → it never skips.

React only skips if:

- Component state didn't change AND
- Parent didn't re-render

But if parent re-renders → child re-renders too (unless memoized).

> **Clear Mental Model**
>
> Re-render triggers:
>
> - Component state change
> - Parent re-render
>
> Skipping only happens with:
>
> - React.memo
> - PureComponent (class)
> - shouldComponentUpdate
>
> **Interview-Level Answer**
>
> By default, when a parent re-renders, all children re-render regardless of whether their props changed. React.memo adds a shallow prop comparison to skip unnecessary child re-renders.

useMemo

- **The problem without `useMemo`**

Every render runs all code again.

```
const filtered = users.filter(u => u.active);
```

If the component renders **10 times**, the filter runs **10 times**.

Even if `users` did not change.

- **What `useMemo` does**

`useMemo` **remembers the result of a calculation**.

```
const filtered = useMemo(() => {
  return users.filter(u => u.active);
}, [users]);
```

Now React does:

```
if users changed
    run filter again
else
    reuse previous result
```

React stores the **value**, not the function.

Example stored internally:

```
previousUsers → reference
previousResult → filtered array
```

If dependencies are the same → return **previousResult**.

> `useMemo` memoizes a computed value and recalculates it only when its dependencies change, preventing unnecessary recalculations and helping keep stable references.

useCallback

Every time a component renders, **JavaScript creates new functions**.

Example:

```
<MyComp onClick={() => doSomething()} />
```

When the parent renders again:

- React executes the component again
- `() => doSomething()` is **created again**
- It is a **new reference in memory**

So React sees:

```
old onClick !== new onClick
```

Even if the logic is identical.

Why this matters

If the child component is optimized (for example with `React.memo`):

```
const MyComp = React.memo(({ onClick }) => {
  console.log("render");
  return <button onClick={onClick}>Click</button>;
});
```

React compares props.But because the function reference changed:

```
old function !== new function
```

React thinks the prop changed → **child re-renders**.

What `useCallback` does?

`useCallback` **keeps the same function reference** between renders.

```
const handleClick = useCallback(() => {
  doSomething();
}, []);
```

Now React keeps **the same function in memory**.

So next render:

```
old handleClick === new handleClick
```

Child props didn't change → **React.memo can skip re-render**.

> `useCallback` memoizes a function so that its reference stays stable across renders. This helps prevent unnecessary re-renders of memoized child components that receive the function as a prop.

Hook	Think of it as
React.memo	**"Don't re-render this component if props didn't change"**
useMemo	**"Don't recompute this value unless dependencies change"**
useCallback	**"Don't recreate this function unless dependencies change"**

Component Lifecycle (Hooks Version)

Three moments in a component life

Phase	Meaning
Mount	Component appears on the screen for the first time
Update	Component re-renders because **state or props changed**

Unmount	Component is removed from the screen

Example with `useEffect`

```
useEffect(() => {
  console.log("mounted");

  return () => {
    console.log("unmounted");
  };
}, []);
```

Because the dependency array is **empty** `[]`:

- effect runs **once after mount**
- cleanup runs **when component unmounts**

Step by step:

```
Component renders
↓
React puts it in the DOM
↓
useEffect runs → "mounted"
```

Later when the component disappears:

```
React removes component
↓
cleanup function runs → "unmounted"
```

```
Mount → component appears
Update → state / props change
Unmount → component disappears
```

`useEffect` lets you **run code during these moments**.

Context API

The Context API lets you share data across the component tree **without passing props at every level**.

The problem it solves (prop drilling)

```
App  (has user data)
 └── Dashboard
      └── Sidebar
           └── UserAvatar  ← needs user data
```

Without Context, you pass `user` as a prop through Dashboard and Sidebar even though they don't use it. That's prop drilling — messy and hard to maintain.

```jsx
// 1. Create the context
const UserContext = React.createContext();

// 2. Wrap your tree with a Provider
function App() {
  return (
    <UserContext.Provider value={{ name: "Jouhar" }}>
      <Dashboard />
    </UserContext.Provider>
  );
}

// 3. Consume it anywhere with useContext
function UserAvatar() {
  const { name } = useContext(UserContext);
  return <p>{name}</p>;
}
```

> Every component that consumes a context **re-renders when the value changes**. Don't put everything in one context — split by concern (UserContext, ThemeContext, etc).

Custom Hooks

What is a custom hook?

A custom hook is a **JavaScript function that starts with** `use` and calls other hooks inside it. It lets you extract and reuse stateful logic across multiple components.

The problem it solves

Without custom hooks, if two components need the same logic (like fetching data, or tracking window size), you either duplicate the code or mix logic with UI in one messy component.

Simple example — useFetch

jsx

```jsx
// Without custom hook — logic mixed into component
function UserProfile() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch('/api/user')
      .then(r => r.json())
      .then(d => { setData(d); setLoading(false); });
  }, []);

  if (loading) return <p>Loading...</p>;
  return <p>{data.name}</p>;
}
```

```jsx
// With custom hook — clean separation
function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

```

```
  useEffect(() => {
    fetch(url)
      .then(r => r.json())
      .then(d => { setData(d); setLoading(false); });
  }, [url]);

  return { data, loading };
}

// Now reusable in any component
function UserProfile() {
  const { data, loading } = useFetch('/api/user');
  if (loading) return <p>Loading...</p>;
  return <p>{data.name}</p>;
}
```

Rules (same as all hooks)

- Name must start with `use`
- Can only be called at the **top level** of a component or another hook
- Cannot be called inside loops, conditions, or nested functions

---

Redux

You must understand:

- What problem Redux solves
- Global state vs local state
- Store
- Slice
- Action
- Reducer
- Dispatch
- useSelector
- useDispatch
- Async with `createAsyncThunk`

What Problem Redux Solves

Without Redux:

```
App
 ├── Header
 │
 ├── ProductPage
 │
 └── Cart
```

Suppose **Cart count** must appear in:

- Header
- ProductPage
- Cart page

You would pass props like this:

```
App
 ├── Header (cartCount)
 ├── ProductPage (cartCount, setCartCount)
 └── Cart (cartCount)
```

If the app grows:

```
App → Layout → Page → Component → Child → Button
```

You start passing props through many components.

This is called: **Prop drilling**

Redux solves this.

Instead of passing props everywhere:

```
Redux Store
      ↑
All components read from it
```

Global state.

Global state vs local state

Local State	Global State
useState	Redux store
Only inside component	Accessible everywhere
Small UI data	Shared app data

Store

Store = **central state container**

```
Redux Store
{
  user: {...},
  cart: [...],
  theme: "dark"
}
```

Created once in app.

Example:

```
const store = configureStore({
  reducer: {
    user: userSlice.reducer,
    cart: cartSlice.reducer
  }
});
```

Slice

Think of store as: **Single source of truth**

A **slice** = piece of the store.

Example store:

```
store
 ├── user
 ├── cart
 └── products
```

Each part has its own slice.

Example:

```
const cartSlice = createSlice({
  name: "cart",
  initialState: [],
  reducers: {
    addItem(state, action) {
      state.push(action.payload);
    }
  }
});
```

Slice contains:

- state
- reducers
- actions

All in one file.

Action

Action = **message describing what happened**

Example:

```
ADD_PRODUCT
REMOVE_PRODUCT
LOGIN_SUCCESS
```

Structure:

```
{
  type: "cart/addItem",
  payload: product
}
```

It does **not change state**.

It only **describes change**.

Reducer

Reducer = function that **updates state**

Example:

```
addItem(state, action) {
  state.push(action.payload);
}
```

Reducer rules:

✔ Pure function
✔ No API calls
✔ No side effects
✔ Only update state

Redux Toolkit allows mutation syntax because it uses **Immer internally**.

Dispatch

Dispatch = **send action to store**

Example:

```
dispatch(addItem(product));
```

Flow becomes:

```
Component
   ↓
dispatch(action)
   ↓
Reducer updates store
   ↓
React components re-render
```

useSelector

Reads state from Redux.

Example:

```
const cart = useSelector(state => state.cart);
```

Whenever cart changes → component re-renders.

useDispatch

Gets dispatch function.

```
const dispatch = useDispatch();
```

Used to trigger actions.

We will build a simple Cart example

- **Install Redux Toolkit**

```
npm install @reduxjs/toolkit react-redux
```

Redux Toolkit is the **modern Redux** (interviewers expect this).

- **Create the Store**

📁 `store.js`

```
import { configureStore } from "@reduxjs/toolkit";
import cartReducer from "./cartSlice";

export const store = configureStore({
  reducer: {
    cart: cartReducer
  }
});
```

Now the global store looks like:

```
store
 └── cart
```

- **Create a Slice**

📁 `cartSlice.js`

```
import { createSlice } from "@reduxjs/toolkit";

const cartSlice = createSlice({
  name: "cart",

  initialState: [],

  reducers: {

    addItem: (state, action) => {
      state.push(action.payload);
    },

    removeItem: (state, action) => {
      return state.filter(item => item.id !== action.payload);
    }
```

```

  }
});

export const { addItem, removeItem } = cartSlice.actions;

export default cartSlice.reducer;
```

This file automatically creates:

```
Action creators
Reducers
```

Example generated action:

```
{
  type: "cart/addItem",
  payload: product
}
```

- **Connect Redux to React**

📁 `index.js`

```
import React from "react";
import ReactDOM from "react-dom/client";
import { Provider } from "react-redux";
import { store } from "./store";
import App from "./App";

const root = ReactDOM.createRoot(document.getElementById("root"));

root.render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

`Provider` makes Redux store available to all components.

- **Component That Dispatches Action**

📁 `Product.js`

```
import React from "react";
import { useDispatch } from "react-redux";
import { addItem } from "./cartSlice";

function Product() {

  const dispatch = useDispatch();

  const product = {
    id: 1,
    name: "Laptop"
  };

  const handleAdd = () => {
    dispatch(addItem(product));
  };

  return (
```

```
    <div>
      <h2>{product.name}</h2>
      <button onClick={handleAdd}>Add to Cart</button>
    </div>
  );
}

export default Product;
```

Here:

```
Component → dispatch(action)
```

- **Component That Reads State**

📁 `Cart.js`

```
import React from "react";
import { useSelector } from "react-redux";

function Cart() {

  const cart = useSelector(state => state.cart);

  return (
    <div>
      <h2>Cart</h2>

      {cart.map(item => (
        <p key={item.id}>{item.name}</p>
      ))}

    </div>
  );
}

export default Cart;
```

Important:

```
useSelector subscribes to store
```

If cart changes → component re-renders.

Main App

📁 `App.js`

```
import React from "react";
import Product from "./Product";
import Cart from "./Cart";

function App() {

  return (
    <div>
      <h1>Redux Example</h1>

      <Product />
      <Cart />

```

```
    </div>
  );
}

export default App;
```

Full Redux Flow (What Happens When Button Clicks)

```
Component
   ↓
dispatch(action)
   ↓
Reducer updates state
   ↓
Store changes
   ↓
useSelector detects change
   ↓
Component re-renders
```

> A component dispatches an action. The Redux store forwards that action to reducers. Reducers compute a new state. The store updates its state and subscribed components using useSelector re-render with the new data.

Element	Role
Store	holds global state
Action	describes what happened
Reducer	updates state
Component	reads & dispatches

Async with `createAsyncThunk`

Problem

Redux **reducers must be pure**.

That means:

❌ No API calls
❌ No async code

Reducers can **only update state**.

Solution

Redux Toolkit provides: `createAsyncThunk`

It handles **API requests outside reducers**.

Example:

```
export const fetchUsers = createAsyncThunk(
  "users/fetch",
  async () => {
    const response = await fetch("/api/users");
    return response.json();
  }
);
```

What Redux automatically creates

Redux generates **3 actions automatically**:

State	Meaning
pending	request started
fulfilled	request success
rejected	request failed

Example:

```
fetchUsers/pending
fetchUsers/fulfilled
fetchUsers/rejected
```

Reducers handle those states

```
extraReducers: builder => {
  builder
    .addCase(fetchUsers.pending, state => {
      state.loading = true;
    })
    .addCase(fetchUsers.fulfilled, (state, action) => {
      state.users = action.payload;
    })
    .addCase(fetchUsers.rejected, state => {
      state.loading = false;
    });
}
```

Most Common React + Redux Interview Questions

Be ready for these:

What happens when you call dispatch?

Dispatch sends an action to the Redux store. Reducers handle the action and return a new state. The store updates, and components using `useSelector` re-render if the selected data changed.

> Redux **does not update state directly**.
>
> Reducer returns a **new state object**.
>
> React-Redux then uses **reference comparison** to detect changes.

```
Component
   ↓
dispatch(action)
   ↓
Redux store
   ↓
Reducer runs
   ↓
New state returned
   ↓
Store updates
   ↓
useSelector compares values
   ↓
Component re-renders if needed
```

What is immutability and why important?

Immutability means:

> **You never modify existing data. You create a new copy instead.**

Example.

Mutation (BAD)

```
const state = { count: 0 };

state.count = 1; // modifying existing object
```

You changed the **same object**.

Immutable update (GOOD)

```
const state = { count: 0 };

const newState = { ...state, count: 1 };
```

Now:

```
old state ≠ new state
```

Two different objects.

- **Why Redux Requires Immutability**

Redux detects changes using **reference comparison**.

Example.

Old state:

```
state = { count: 0 }
```

If you mutate:

```
state.count = 1
```

The object reference is still the same.

```
state === state
```

Redux thinks:

```
Nothing changed
```

So components **won't re-render correctly**.

- **Correct Redux Update**

Reducer must return a **new object**.

Example reducer:

```
function reducer(state, action) {

  if (action.type === "increment") {
    return {
      ...state,
      count: state.count + 1
    };
  }

  return state;
}
```

New state → new reference → React detects change.

- **Why Redux Toolkit Looks Like Mutation**

You might see this:

```
state.count += 1
```

Looks like mutation.

But Redux Toolkit uses **Immer** internally.

Immer converts it into immutable update.

So internally it becomes something like:

```
return {
  ...state,
  count: state.count + 1
}
```

> Immutability means we never modify the existing state directly. Instead, we return a new copy of the state with the updated values. Redux relies on reference comparison to detect changes, so mutating the state can break change detection and prevent components from updating correctly.

How to avoid unnecessary re-renders?

We can avoid unnecessary re-renders using `React.memo`, `useMemo`, and `useCallback`. `React.memo` prevents a component from re-rendering if its props did not change using shallow comparison.
`useMemo` memoizes expensive calculations to avoid recomputing values on every render.
`useCallback` memoizes functions so their reference stays stable when passed to child components.

How to structure a React project?

Typical React Project Structure

```
src/
  components/
  pages/
  store/
    slices/
  services/
  hooks/
  utils/
  App.js
  index.js
```

Now let's explain each folder **like in an interview**.

components/

Reusable UI components.

Example:

```
components/
  Button.jsx
  Modal.jsx
  Input.jsx
  Navbar.jsx
```

These components:

- are **generic**
- can be reused in many pages
- usually contain **UI only**

Example:

```
<Button />
<Input />
<Card />
```

Pages/

Pages correspond to **routes**.

Example:

```
pages/
  HomePage.jsx
  ProductPage.jsx
  CartPage.jsx
  LoginPage.jsx
```

Pages:

- combine multiple components
- represent **screens**

Example:

```
ProductPage
  ├ ProductList
  ├ Filters
  └ Pagination
```

store/

Redux logic.

Example:

```
store/
  store.js
  slices/
    cartSlice.js
    userSlice.js
```

Each slice manages **one part of global state**.

Example:

```
store
 ├ cart
```

```
  └ user
```

services/

API calls.

Example:

```
services/
  api.js
  userService.js
  productService.js
```

Example code:

```
export const fetchProducts = () =>
  axios.get("/api/products");
```

Keeps API logic **separate from UI**.

hooks/

Custom React hooks.

Example:

```
hooks/
  useAuth.js
  useCart.js
  useFetch.js
```

Example:

```
function useAuth() {
  const user = useSelector(state => state.user);
  return user;
}
```

Helps **reuse logic across components**.

utils/

Helper functions.

Example:

```
utils/
  formatPrice.js
  debounce.js
  validators.js
```

Example:

```
export function formatPrice(value) {
  return "$" + value.toFixed(2);
}
```

index.js

Entry point of React app.

Example:

```
ReactDOM.createRoot(document.getElementById("root")).render(
  <Provider store={store}>
    <App />
  </Provider>
);
```

App.js

Main component.

Usually contains:

- routing
- layout

Example:

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<HomePage />} />
    <Route path="/cart" element={<CartPage />} />
  </Routes>
</BrowserRouter>
```